

USER MANUAL

AI Audio Mixing & Mastering Analyzer

1. Overview

This standalone Windows-oriented Python desktop application analyzes finished mixes, masters, stems and reference material without modifying source audio. It produces measurements, warnings and AI-ready evidence for mixing and mastering decisions.

The application is an analyzer and decision-support tool. It does not automatically change audio, apply EQ, compress files, or replace critical listening.

2. Installation

Install Python 3.10 or newer. Open a terminal in the application folder and run:

```
pip install -r requirements.txt
```

Start the program with:

```
python main.py
```

Required packages include NumPy, SciPy, SoundFile, Matplotlib and Pytest.

3. Uploading files

Click **UPLOAD FILES** and select one or more WAV, AIFF, FLAC, OGG or MP3 files supported by the installed SoundFile backend. Original files are read only.

Uploaded files appear in the file list. Select a file after analysis to inspect its individual results.

4. Running analysis

Click **ANALYZE ALL**. Files are processed in the background so the interface remains responsive. The progress bar shows batch progress.

If a file cannot be decoded, its error is recorded while other files continue.

5. Reading results

MIX SCORE is an explainable screening score, not an artistic quality rating. LUFS indicates integrated loudness. TRUE PEAK estimates inter-sample peak level. RMS describes average signal energy. CORRELATION indicates left/right similarity and possible mono compatibility risk. CLIPS reports samples at or above digital full scale.

The right-hand **AI-READY FINDINGS** panel lists severity, measured evidence, what to check and confidence.

6. AI BRIEF export

Click **AI BRIEF** to export a JSON package containing every analyzed file, measurements, findings, target loudness and instructions for an AI assistant. Upload this JSON to an AI system and ask it to prioritize mix decisions using evidence only.

7. Recommended workflow

Analyze the mix before and after each major revision. Compare LUFS and true peak at matched playback levels. Investigate warnings in the DAW with solo, spectrum, correlation and mono checks. Make small changes, export a new version, and re-analyze.

TECHNICAL REFERENCE

AI Audio Mixing & Mastering Analyzer

1. Processing architecture

The interface is implemented in Tkinter. Analysis runs in a worker thread and communicates progress back to the GUI. DSP is separated into analyzer modules; AI interpretation and reporting are separate modules.

Main pipeline: audio_loader -> loudness, true_peak, spectrum, stereo, quality -> interpreter
-> JSON/CSV/AI brief.

2. Audio loading

audio_loader.py uses SoundFile to decode supported formats and converts samples to float32 without normalization. Metadata includes filename, duration, sample rate, channel count and source subtype. Source files are never overwritten.

3. Loudness

loudness.py applies high-pass and high-shelf K-weighting approximations, creates overlapping loudness windows, records loudness history and calculates integrated, momentary, short-term and LRA-style values.

For mastering release decisions, validate results against a trusted BS.1770/EBU R128 meter because this foundation is an approximation and should not yet be treated as a certified compliance meter.

4. True peak

true_peak.py uses SciPy resample_poly oversampling, currently at 4x, and reports the maximum reconstructed sample level in dBTP plus its approximate source position.

5. Spectrum

spectrum.py computes a Hann-windowed real FFT, reports frequency bins, magnitude, centroid, bandwidth and energy in seven bands: sub-bass, bass, low-mid, mid, upper-mid, presence and high.

6. Stereo and phase

stereo.py calculates left/right RMS, Mid and Side RMS, width ratio and inter-channel correlation. Negative correlation is flagged as a potential phase/mono compatibility issue. It is a diagnostic indicator, not proof of an audible defect.

7. Quality checks

quality.py reports sample peak, clipped sample count and timestamps, per-channel DC offset, RMS and crest factor. Timestamp lists are capped at the first 100 events to keep reports manageable.

8. AI safety model

interpreter.py produces structured findings with problem, evidence, severity, explanation, suggested checks and confidence. It must not invent measurements. Frequency and stereo findings are phrased as possible issues requiring listening and engineering verification.

9. Export schema

Per-file JSON contains file metadata plus loudness, true_peak, spectrum, stereo and quality